

DETECTION ENGINEERING REPORT

**Custom Wazuh Detection Logic for Network Reconnaissance, Obfuscated Execution,
Persistence, and Web Shell Deployment**

Wazuh SIEM • Suricata IDS • Linux Auditd

Prepared by: Amaima
Detection Engineering Lab

Environment: Wazuh Manager (Ubuntu) – Wazuh Agent (Kali Linux) – Suricata 8.0.5 – auditd

1. Executive Summary

This report documents the design, implementation, and validation of four custom Wazuh detection rules developed to close visibility gaps not addressed by Wazuh's default ruleset. The detection engineering effort targets behaviors observed across the reconnaissance, defense evasion, persistence, and initial-access stages of a typical attack chain, using telemetry generated by Suricata IDS and Linux auditd within a controlled lab environment.

The lab consists of a Wazuh Manager running on Ubuntu, a Wazuh Agent deployed on a Kali Linux host, Suricata 8.0.5 operating as a network intrusion detection sensor, and auditd providing host-level behavioral monitoring on the monitored Linux endpoint. All source telemetry is forwarded into the Wazuh Manager, where custom rules defined in `local_rules.xml` correlate raw log data into actionable security alerts.

Four detection scenarios were implemented and validated:

- Nmap SYN Scan Detection – network reconnaissance via Suricata `eve.json` (MITRE T1046)
- Base64 Encoded Command Execution Detection – defense evasion via auditd (MITRE T1027)
- Suspicious User Creation Detection – persistence via auditd (MITRE T1136)
- Web Shell Upload Detection – initial access / persistence via auditd (MITRE T1505.003)

Each detection was engineered following a repeatable methodology: define the threat scenario, simulate the attacker behavior in a controlled manner, capture raw source telemetry, analyze how Wazuh decodes that telemetry, author a custom rule, and validate that the rule produces an alert consistent with the underlying evidence. Throughout this report a clear distinction is maintained between telemetry generated at the source (Suricata or auditd) and the alert generated by Wazuh after rule evaluation – the two are not the same artifact, and conflating them is a common documentation error this report deliberately avoids.

Results indicate that all four rules functioned as designed within the lab environment, generating alerts at appropriate severity levels with contextual MITRE ATT&CK mapping. Results are reported as observed; no detection is claimed to be complete or production-hardened, and known limitations and tuning recommendations are discussed in Section 6.

2. Lab Architecture

The lab was built to reflect a minimal but representative SOC data pipeline: a network sensor, a host-level auditing agent, and a central correlation engine. The architecture below describes each component's role and how log data moves between them.

2.1 Components

Component	Role	Detail
Wazuh Manager	Central correlation engine	Ubuntu host; receives, decodes, and evaluates rules against all forwarded logs
Wazuh Agent	Endpoint telemetry collector	Deployed on Kali Linux; forwards auditd and local log data to the Manager
Suricata IDS	Network-layer detection sensor	Version 8.0.5; inspects network traffic and writes alerts to <code>eve.json</code>

Component	Role	Detail
Linux auditd	Host-level behavioral monitoring	Custom audit rules with descriptive keys (e.g. base64_exec, user_creation, webroot_changes)
local_rules.xml	Custom detection logic	Stores all rules authored for this project, decoupled from Wazuh's default ruleset

2.2 Data Flow

The data flow follows a linear pipeline from telemetry generation to alerting:

1. Suricata inspects network traffic on the monitored segment and writes structured JSON alerts to eve.json when a signature matches (e.g. signature_id 9000001 for the custom Nmap SYN scan signature).
2. auditd monitors specific system calls and filesystem paths on the Kali Linux endpoint according to rules tagged with a descriptive key (base64_exec, user_creation, webroot_changes), writing matching events to the Linux audit log.
3. The Wazuh Agent installed on the Kali Linux host tails both the Suricata eve.json file and the local audit log, forwarding new events to the Wazuh Manager over the encrypted agent-manager channel.
4. The Wazuh Manager receives the forwarded events and passes them through its decoding pipeline, which parses the raw log into structured fields (see Section 3).
5. Decoded events are evaluated against the active ruleset, which includes both Wazuh's default rules and the custom rules defined in local_rules.xml for this project.
6. When a rule condition is satisfied, Wazuh generates an alert containing the rule ID, severity level, description, and MITRE ATT&CK mapping, which is written to the alerts log and surfaced in the Wazuh dashboard.

Conceptually: Attacker Action → Suricata / auditd (source telemetry) → Wazuh Agent (forwarding) → Wazuh Manager Decoder → Wazuh Rule Engine → Wazuh Alert (dashboard / alerts.json).

3. How Wazuh Processes Logs

Understanding Wazuh's internal processing pipeline is a prerequisite for writing correct custom rules. Each stage below directly affects how the four detections in this report were designed.

3.1 Log Collection

The Wazuh Agent collects logs from configured sources on the monitored endpoint. In this lab, two sources are relevant: the Suricata eve.json file (read as a JSON log source) and the Linux audit log populated by auditd (read via the audit log connector). The agent forwards these events, largely unmodified, to the Manager over the agent-manager protocol.

3.2 Decoders

On the Manager, incoming events first pass through the decoder stage. A decoder's job is to parse an unstructured or semi-structured log line into named fields that rules can reference. For JSON-formatted Suricata alerts, Wazuh applies its built-in JSON decoding logic, extracting fields such as data.alert.signature_id, data.src_ip, data.dest_ip, and data.proto directly from the eve.json structure. For auditd events, Wazuh uses its built-in auditd decoder, which parses the audit record type (e.g. SYSCALL, EXECVE, PATH) and extracts fields such as data.audit.key, data.audit.exe, data.audit.command, and data.audit.execve.a0. Correct field extraction at this stage is what allows subsequent rules to match on specific values rather than performing brittle raw-text pattern matching.

3.3 Rules and Rule Hierarchy

Rules are evaluated after decoding, in a tree structure. A rule can be a root rule that matches on decoder output directly, or a child rule that uses `if_sid` to inherit from a parent rule and add further matching conditions. This lab's custom rules follow the standard pattern of a low-level base rule (which simply confirms the relevant decoder and key/field matched) followed by one or more child rules that add specificity – such as a particular audit key, a particular signature_id, or a frequency/timeframe condition. This hierarchy keeps individual rules simple and composable, and avoids duplicating match logic across similar detections.

3.4 Rule Levels

Every Wazuh rule declares a severity level from 0 to 16. Levels are not arbitrary; they are meant to communicate operational urgency to an analyst. In this project, levels were assigned using the following reasoning: low levels (0–4) for informational or benign audit activity; medium levels (5–8) for behavior that is suspicious but has legitimate administrative explanations (e.g. base64 decoding, account creation); high levels (9–12) for behavior with a narrow legitimate use case and a high likelihood of malicious intent (e.g. a web shell written into a web root, or a scan pattern repeated across many ports). No rule in this project was assigned above level 12, since none of the four scenarios represents confirmed, high-confidence compromise on its own – each is a strong indicator that should still be triaged by an analyst.

3.5 Frequency and Timeframe

The frequency and timeframe rule options allow a rule to require that a condition be met a minimum number of times within a bounded time window before an alert fires. This is the primary mechanism used in this project to separate noisy, low-confidence single events (e.g. one SYN packet, one base64 command) from a pattern consistent with actual scanning or scripted attacker behavior. Frequency-based correlation is discussed in more detail within each detection's rule logic in Section 4 and again in Section 6.

3.6 MITRE ATT&CK Mapping

Wazuh supports attaching MITRE ATT&CK technique IDs directly to a rule using the `<mitre><id>` tag. This does not affect matching logic; it is metadata that enriches the resulting alert so analysts can immediately see which stage of the attack lifecycle a given alert corresponds to. Every custom rule in this project includes an explicit MITRE mapping for this reason.

4. Detection Engineering Methodology

The following four subsections document each detection scenario using a consistent structure: threat scenario, data source, attack simulation steps, raw evidence, decoder analysis, custom rule logic, alert output, MITRE ATT&CK mapping, severity selection, and false-positive reduction. This structure is intended to make each detection independently auditable and reproducible.

4.1 Detection 1 – Nmap SYN Scan Detection

A. Threat Scenario

Network reconnaissance is typically the first phase of an external or lateral attack. An attacker with network access uses a SYN (half-open) scan to enumerate live hosts and open TCP ports without completing the full TCP handshake, reducing

the likelihood of the connection being logged by application-layer services. In this lab, an attacker positioned on the network (192.168.0.122) performs a SYN scan against a target host (192.168.0.113) to identify open services prior to exploitation.

B. Data Source

Suricata eve.json, specifically the alert event type generated when Suricata's signature engine matches TCP packets consistent with a SYN scan pattern (multiple SYN packets to varied destination ports without completed handshakes, or a custom signature written for this behavior).

C. Attack Simulation Steps

1. From the attacker host (192.168.0.122), run an Nmap SYN scan against the target: `nmap -sS 192.168.0.113`
2. Suricata, running in inline or IDS mode on the monitored interface, inspects the resulting TCP traffic against its loaded signature set, including the custom signature authored for this project (SID 9000001).
3. Suricata writes a JSON alert record to eve.json when the signature condition is satisfied.
4. The Wazuh Agent forwards the new eve.json entry to the Wazuh Manager.

D. Raw Evidence (Suricata eve.json – source telemetry)

```
{
  "event_type": "alert",
  "signature_id": 9000001,
  "signature": "CUSTOM Possible Nmap SYN Scan Detected",
  "src_ip": "192.168.0.122",
  "dest_ip": "192.168.0.113",
  "proto": "TCP"
}
```

Note: this is source telemetry generated by Suricata. It confirms that Suricata's signature matched – it does not, by itself, confirm that Wazuh generated an alert. That determination depends on whether the custom Wazuh rule below also matched after decoding.

E. Decoder Analysis

Wazuh applies its built-in JSON decoder to the eve.json line, which is exposed to the rule engine as a set of dot-notation fields, most relevantly:

- data.alert.signature_id → 9000001
- data.alert.signature → CUSTOM Possible Nmap SYN Scan Detected
- data.src_ip → 192.168.0.122
- data.dest_ip → 192.168.0.113
- data.proto → TCP

No custom decoder was required for this detection, since Wazuh's default eve.json / Suricata decoding logic already exposes signature_id as a matchable field.

F. Custom Rule Logic (local_rules.xml)

```
<group name="suricata,recon,">

  <!-- Base rule: confirm Suricata alert decoder fired -->
  <rule id="100010" level="3">
    <if_sid>86601</if_sid>
    <field name="alert.signature_id">9000001</field>
```

```

<description>Suricata: Custom Nmap SYN scan signature matched</description>
</rule>

<!-- Correlation rule: repeated matches from same source within window -->
<rule id="100011" level="10" frequency="5" timeframe="120">
  <if_matched_sid>100010</if_matched_sid>
  <description>Possible Nmap SYN scan in progress from $(srcip)</description>
  <mitre>
    <id>T1046</id>
  </mitre>
  <group>network_scan,recon,</group>
</rule>

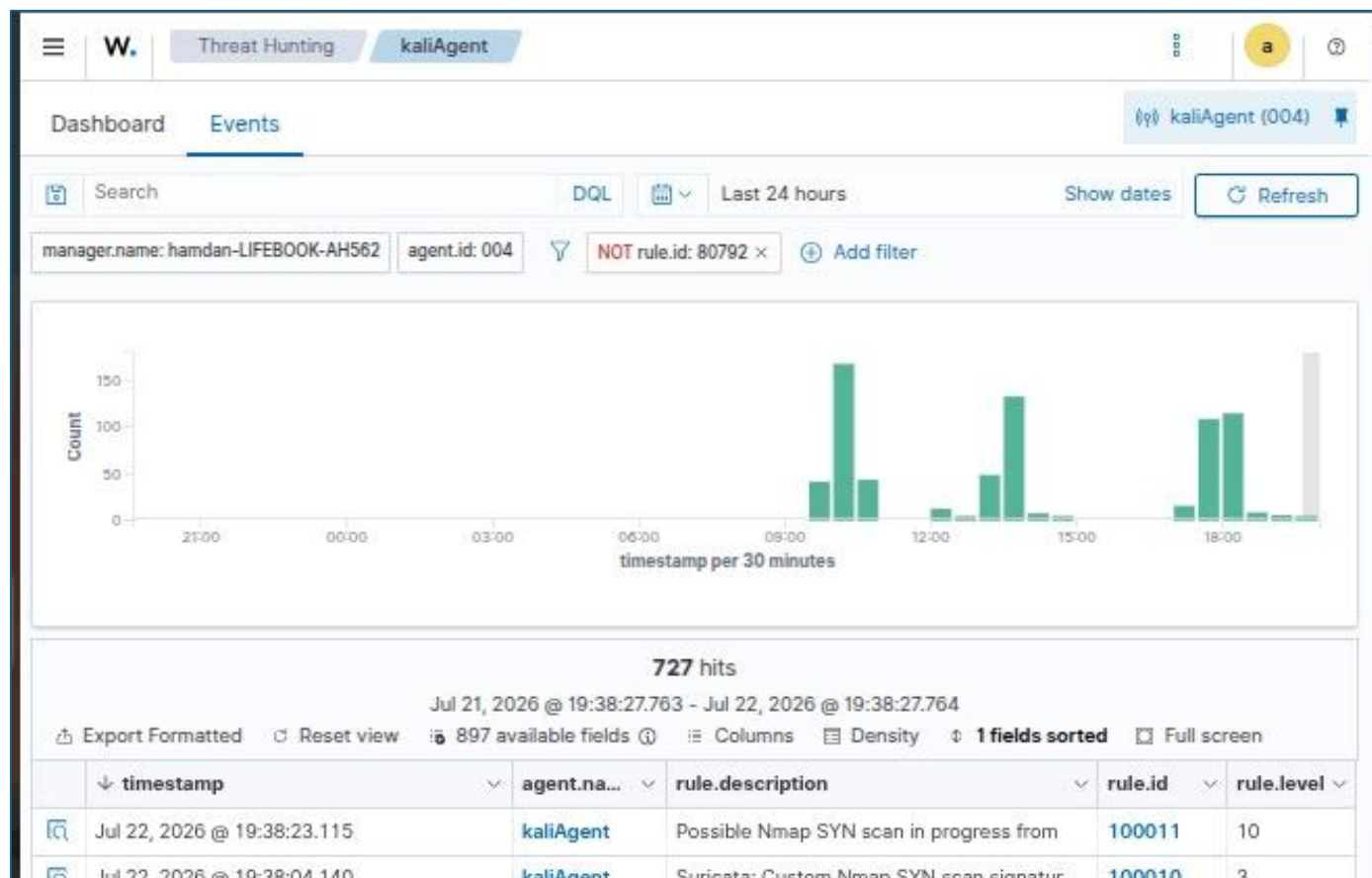
</group>

```

Rule 100010 is intentionally low-level (level 3): it only confirms that the specific custom Suricata signature fired, and is not intended to alert an analyst on its own. Rule 100011 is the operationally meaningful rule – it requires the same base condition to repeat at least five times from the same source IP within a 120-second window before escalating to level 10. This two-tier structure follows the rule hierarchy described in Section 3.3 and reflects the reasoning that a single SYN packet match is common background noise, while a burst of matches from one source is consistent with an active port sweep.

G. Alert Output (Wazuh – generated alert)

When the frequency condition is satisfied, Wazuh generates an alert referencing rule 100011, including the source and destination IPs, protocol, rule level 10, and the associated MITRE technique. This is the Wazuh-generated alert, distinct from the raw Suricata telemetry shown in part D – it only exists because the correlation condition in rule 100011 was met.



H. MITRE ATT&CK Mapping

Tactic: Reconnaissance

Technique: T1046 – Network Service Scanning

SYN scanning is a direct implementation of network service scanning: the attacker is enumerating which services are reachable and listening on the target before selecting an exploitation path.

I. Severity Justification

Rule 100010 is set to level 3 because a single signature match, in isolation, does not indicate an active attack – it could reflect a single misdirected packet or an authorized scan. Rule 100011 is set to level 10 because a sustained pattern of matches from one source is a strong reconnaissance indicator that typically precedes a more damaging action; level 10 sits in Wazuh's "high severity" band without treating the event as confirmed compromise, since scanning alone does not establish that exploitation followed.

J. False-Positive Reduction

- Frequency/timeframe correlation (5 matches / 120 seconds) filters out isolated or coincidental single-packet matches.
- `same_source_ip` binding ensures the correlation only fires when the repeated activity originates from one host, rather than aggregating unrelated traffic.
- Restricting the base rule to the specific custom signature_id (9000001) avoids matching on Suricata's broader, noisier default scan signatures.
- Known internal vulnerability scanners or monitoring tools should be added to an allow-list by source IP at the rule level (see Section 6) to avoid recurring false positives from authorized scanning.

4.2 Detection 2 – Base64 Encoded Command Execution Detection

A. Threat Scenario

After gaining initial access, an attacker frequently needs to execute commands or scripts while avoiding detection by security tooling and evading simple string-based content inspection. A common technique is to base64-encode a shell command or payload and decode it at execution time (e.g. `echo <base64> | base64 -d | bash`), which conceals the literal command text from casual log review and some pattern-matching defenses.

B. Data Source

Linux auditd, via a custom audit rule tagged with the key `base64_exec`, which monitors `execve` system calls associated with the `base64` utility and shell interpreters.

C. Attack Simulation Steps

1. On the monitored Kali Linux host, an attacker (or simulated attacker) runs a command such as: `echo 'd2hvYW1p' | base64 -d | bash`
2. The auditd rule configured with `-k base64_exec` captures the resulting `execve` system call, including the full command line.
3. auditd writes the event to the local audit log.
4. The Wazuh Agent forwards the audit event to the Wazuh Manager.

D. Raw Evidence (auditd – source telemetry)

```
type=SYSCALL msg=audit(1737200000.123:4521): arch=c000003e syscall=59
  success=yes exit=0 a0=... comm="base64" exe="/usr/bin/base64"
  key="base64_exec"
type=EXECVE msg=audit(1737200000.123:4521): argc=2 a0="base64" a1="-d"
```

This is source telemetry generated directly by auditd. It confirms the `base64` utility was executed with the monitored key – it does not by itself confirm a Wazuh alert was raised, which depends on the rule match described below.

E. Decoder Analysis

Wazuh's built-in auditd decoder parses the SYSCALL and EXECVE records into structured fields, most relevantly:

- `data.audit.key` → `base64_exec`
- `data.audit.exe` → `/usr/bin/base64`
- `data.audit.command` / `data.audit.execve.a0` → `base64`
- `data.audit.execve.a1` → `-d`

These fields allow the custom rule to match specifically on the audit key rather than parsing the raw audit line with regular expressions, which keeps the rule readable and less brittle against minor audit log formatting differences.

F. Custom Rule Logic (local_rules.xml)

```
<group name="auditd,defense_evasion,">

  <rule id="100020" level="7">
    <if_sid>80700</if_sid>
    <field name="audit.key">base64_exec</field>
    <description>Possible base64-encoded command execution detected ($(audit.exe))</description>
    <mitre>
      <id>T1027</id>
    </mitre>
  </rule>
</group>
```



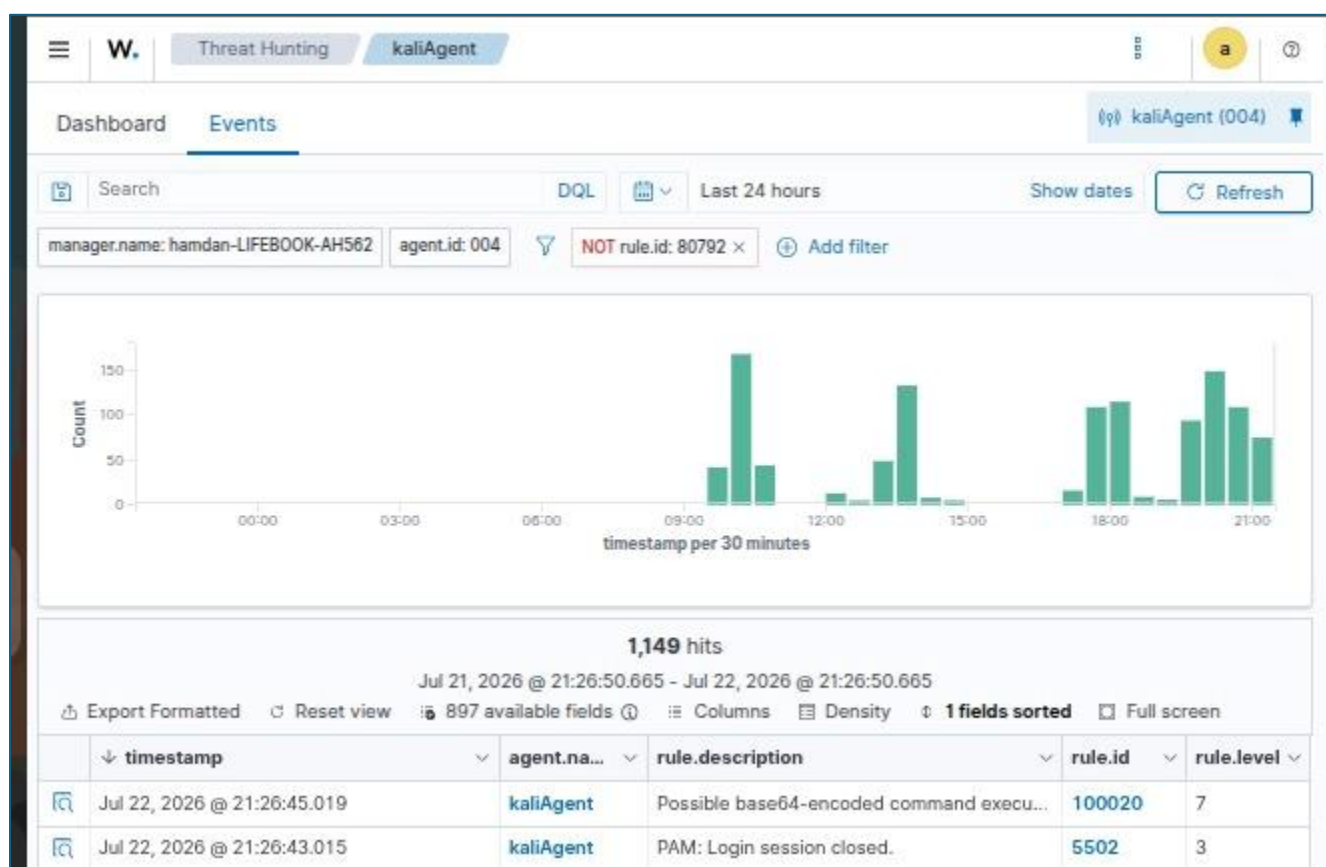
```
<group>obfuscation,base64,</group>
</rule>

</group>
```

This detection uses a single-tier rule rather than a frequency-based pair, because a single instance of piping base64 output into a shell interpreter is already a meaningful indicator on most non-administrative endpoints – unlike a network scan, there is no benign burst pattern to distinguish from noise. `if_sid 80792` refers to Wazuh's default auditd base rule group, ensuring this custom rule only evaluates events that already passed successfully through auditd decoding.

G. Alert Output (Wazuh – generated alert)

When an `execve` event carries `audit.key = base64_exec`, Wazuh raises an alert for rule 100020 at level 7, including the executable path and command context. This is the Wazuh-generated alert; the underlying audit record shown in part D is the source telemetry that made the alert possible.



H. MITRE ATT&CK Mapping

Tactic: Defense Evasion

Technique: T1027 – Obfuscated Files or Information

Base64 encoding a command prior to execution is a textbook example of obfuscating the content of an action to reduce the likelihood of detection or immediate human comprehension during log review.

I. Severity Justification

Level 7 was chosen because base64 usage has a wide range of legitimate administrative uses (encoding configuration data, certificates, or scripted deployment artifacts), so the behavior alone does not warrant a high-severity classification.

Level 7 places the alert in the medium band – visible and worth analyst review – without asserting confirmed malicious intent from a single audit event.

J. Tuning Methods to Avoid Legitimate Administrative Usage

- Scope the underlying audit rule to specific paths or user contexts where base64 piping into a shell is not an expected administrative pattern, rather than auditing all base64 invocations system-wide.
- Exclude known configuration-management tooling (e.g. Ansible, cloud-init) by process ancestry, since these tools legitimately decode base64 payloads as part of normal operation.
- Consider a secondary rule that specifically flags base64 output piped directly into a shell interpreter (bash, sh, python) rather than alerting on base64 invocation alone, since redirection into an interpreter is a stronger indicator than decoding in isolation.
- Maintain an allow-list of administrator accounts or service accounts with a documented, recurring need to decode base64 content, and suppress or downgrade alerts tied to those accounts.

4.3 Detection 3 – Suspicious User Creation Detection

A. Persistence Technique

Creating a new local account is one of the most durable persistence techniques available to an attacker with sufficient privilege: it survives reboots, does not depend on maintaining an existing shell or process, and can be used to re-enter the system through a normal login path rather than an exploit, reducing the chance of triggering exploit-specific detections on subsequent access.

B. Data Source

Linux auditd, via a custom audit rule tagged with the key `user_creation`, monitoring `execve` calls to `/usr/sbin/useradd`.

C. Attack Simulation Steps

1. On the monitored Kali Linux host, an attacker with sufficient privilege runs: `useradd -m backupsvc`
2. auditd, configured to watch `execve` calls to `/usr/sbin/useradd` under the key `user_creation`, captures the syscall and the associated `EXECVE` argument list.
3. auditd writes the `SYSCALL` and `EXECVE` records to the local audit log.
4. The Wazuh Agent forwards the audit event to the Wazuh Manager.

D. Raw Evidence (auditd – source telemetry)

```
type=SYSCALL msg=audit(1737201500.456:4610): arch=c000003e syscall=59
  success=yes exit=0 exe="/usr/sbin/useradd" key="user_creation"
type=EXECVE msg=audit(1737201500.456:4610): argc=3
a0="useradd" a1="-m" a2="backupsvc"
```

This is source telemetry generated by auditd confirming that the `useradd` binary was executed. Whether this produces a Wazuh alert depends on the custom rule described below matching the decoded audit key.

E. Decoder Analysis

Wazuh's auditd decoder exposes the relevant fields as:

- `data.audit.key` → `user_creation`
- `data.audit.exe` → `/usr/sbin/useradd`
- `data.audit.execve.a0` → `useradd`
- `data.audit.execve.a2` → `backupsvc` (the new account name)

The custom rule matches on the audit key and executable path together, rather than the account name, since account naming conventions vary by environment and are not a reliable universal signal.

F. Custom Rule Logic (local_rules.xml)

```
<group name="auditd,persistence,">

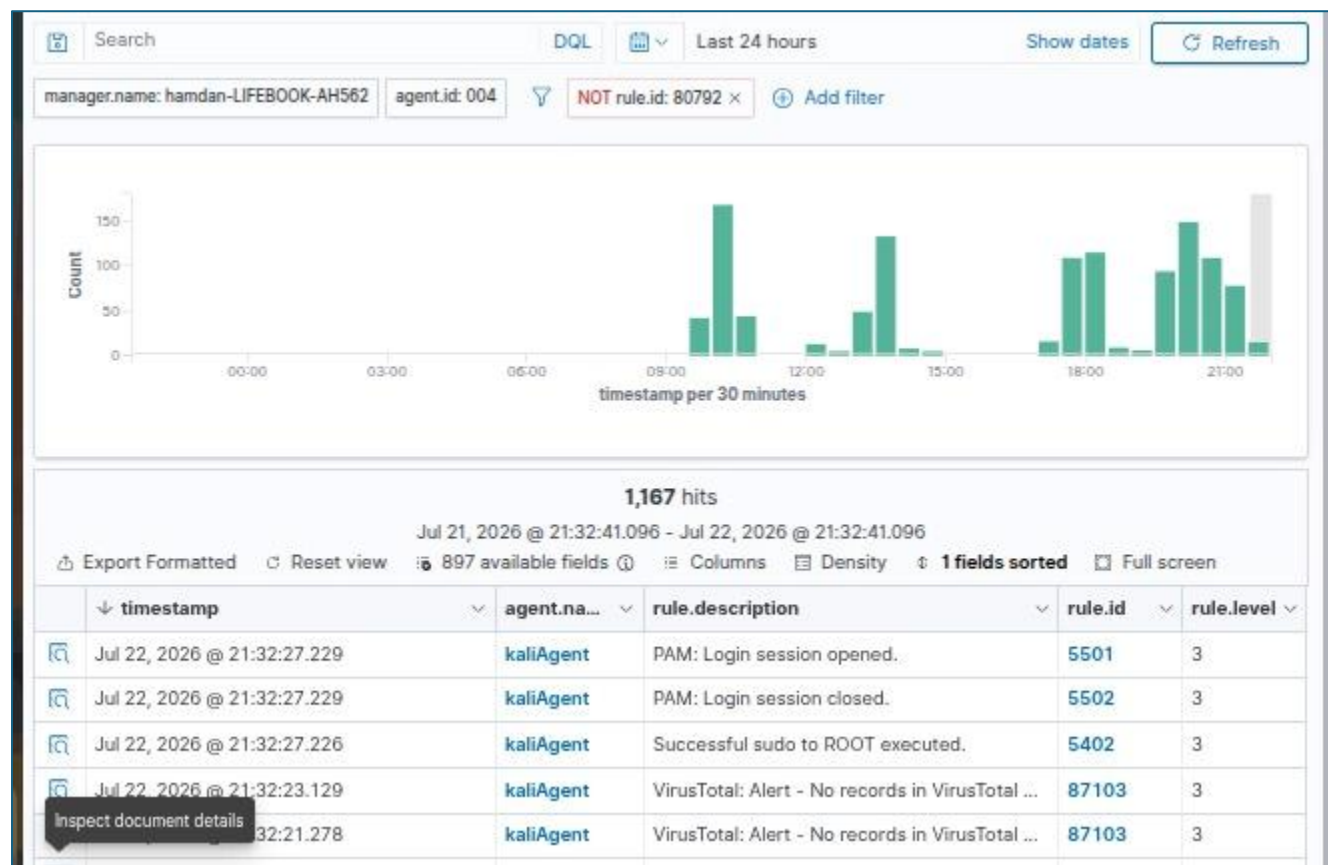
  <rule id="100030" level="8">
    <if_sid>80792</if_sid>
    <field name="audit.key">user_creation</field>
    <field name="audit.exe">/usr/sbin/useradd</field>
    <description>New local user account created via useradd</description>
    <mitre>
      <id>T1136</id>
    </mitre>
    <group>persistence,account_creation,</group>
  </rule>
```

</group>

This rule intentionally does not use frequency correlation: account creation is a discrete, low-volume event on most endpoints, so a single occurrence is already worth surfacing to an analyst rather than waiting for a repeated pattern that may never occur.

G. Alert Output (Wazuh – generated alert)

On matching, Wazuh raises an alert for rule 100030 at level 8, identifying the executable, the audit key, and the new account name extracted from the EXECVE record. This alert is the Wazuh-side artifact; the raw audit lines in part D are the source telemetry that fed it.



H. MITRE ATT&CK Mapping

Tactic: Persistence

Technique: T1136 – Create Account

Unauthorized account creation directly maps to the Create Account technique, which adversaries use to establish a durable foothold independent of any single exploited vulnerability or open session.

I. False-Positive Reduction

- Correlate the alert with the identity of the account performing the creation (auid / uid fields from the audit record) and compare against a known list of authorized system administrators.
- Time-of-day / change-window correlation: account creation outside of documented maintenance windows is a stronger signal than creation during scheduled administrative activity.

- Chain this rule with Detection 3's context to a subsequent privilege-escalation or group-membership-change event (e.g. `usermod -aG sudo`) to prioritize triage of accounts that are both newly created and granted elevated privileges.
- Exclude configuration-management automation (e.g. provisioning scripts run by a known service account during initial host build) via process ancestry or a documented allow-list rather than suppressing the rule outright.

4.4 Detection 4 – Web Shell Upload Detection

A. Attack Chain

A web shell is a script placed within a web server's document root that gives an attacker remote command execution through ordinary HTTP requests, without needing to maintain a separate reverse shell or persistent network connection. The typical chain is: attacker identifies a vulnerable upload function, file-write primitive, or misconfiguration in the web application → writes a script file into the web root → accesses that file directly via a browser or HTTP client to execute attacker-supplied commands through the web server's own process.

B. Data Source

Linux auditd, via a custom audit rule tagged with the key `webroot_changes`, watching for file-write activity under the Apache document root (`/var/www/html`).

C. Attack Simulation Steps

1. On the monitored Kali Linux host, an attacker (simulating a file-write primitive obtained through the web application) writes a PHP file into the web root: `echo '<?php system($_GET["cmd"]); ?>' | tee /var/www/html/shell_test.php`
2. auditd, configured to watch write access under `/var/www/html`, captures the resulting PATH and SYSCALL records under the key `webroot_changes`.
3. auditd writes the event, including the created file name and the process responsible (tee), to the local audit log.
4. The Wazuh Agent forwards the audit event to the Wazuh Manager.

D. Raw Evidence (auditd – source telemetry)

```
type=PATH msg=audit(1737202800.789:4715): item=1
name="/var/www/html/shell_test.php" nametype=CREATE
type=SYSCALL msg=audit(1737202800.789:4715): arch=c000003e syscall=257
success=yes exit=3 exe="/usr/bin/tee" key="webroot_changes"
```

This is source telemetry generated by auditd confirming a file was created under the monitored web root by the tee process. As with the other detections, this does not itself confirm a Wazuh alert – that depends on the rule match below.

E. Decoder Analysis

Wazuh's auditd decoder exposes the relevant fields as:

- `data.audit.file.name` → `/var/www/html/shell_test.php`
- `data.audit.file.nametype` → `CREATE`
- `data.audit.exe` → `/usr/bin/tee`
- `data.audit.key` → `webroot_changes`

The custom rule combines the audit key with a path match ending in `.php`, so that the alert distinguishes a newly created PHP script specifically (the highest-risk file type in this context) from other, lower-risk file writes under the same directory (such as static assets or log files).

F. Custom Rule Logic (local_rules.xml)

```
<group name="auditd,initial_access,web,">

  <!-- Base rule: any tracked write under the web root -->
  <rule id="100040" level="5">
```

```

<!-- Escalation: newly created PHP file specifically -->
<rule id="100041" level="12">
  <if_sid>100040</if_sid>
  <field name="audit.file.nametype">CREATE</field>
  <field name="audit.file.name" type="pcre2">\.php$</field>
  <description>Possible PHP web shell created in web root: ${audit.file.name}</description>
  <mitre>
    <id>T1505.003</id>
  </mitre>
  <group>web_shell,initial_access,</group>
</rule>

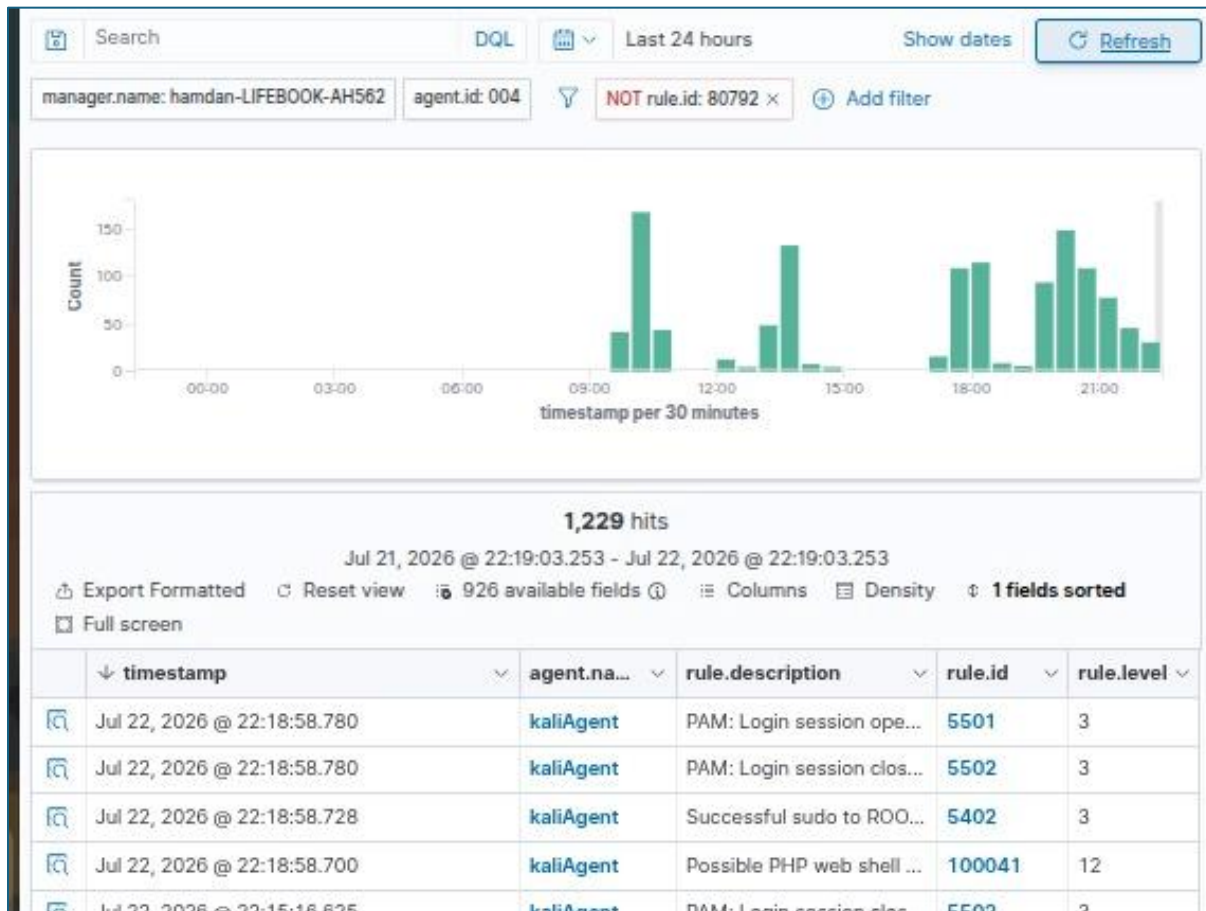
</group>

```

This detection also follows the two-tier hierarchy from Section 3.3: rule 100040 confirms baseline write activity under the monitored directory at low severity, while rule 100041 escalates specifically when a new .php file is created, which is the scenario matching the verified evidence for this project.

G. Alert Output (Wazuh – generated alert)

On matching rule 100041, Wazuh raises a level 12 alert identifying the created file path, the responsible executable, and the MITRE mapping. This is the Wazuh-generated alert; the PATH/SYSCALL records in part D are the underlying source telemetry.



H. MITRE ATT&CK Mapping

Tactic: Persistence / Initial Access

Technique: T1505.003 – Server Software Component: Web Shell

Placing an executable script inside a web-accessible directory to obtain command execution through the web server process is the defining behavior of the Web Shell sub-technique.

I. Why Monitoring Web Directories Is Important

The web document root is one of the highest-value targets for an attacker with any file-write capability against a web application, because a single file placed there can provide durable, network-reachable command execution independent of the original vulnerability used to write it. Unlike many other persistence locations, files in a public web root do not require further privilege escalation or a maintained session to be useful to the attacker – they only require the file to remain in place and reachable over HTTP. Monitoring write and create events specifically in this directory, rather than relying solely on web application logs, provides detection coverage even when the initial write occurs through a vulnerability the application's own logging does not capture (e.g. a path traversal or insecure deserialization bug).

J. False-Positive Reduction

- Restrict monitoring to file creation and modification events (nametype CREATE / rather than read access), since read access to the web root is normal and constant.
- Match specifically on script-executable extensions relevant to the deployed application stack (.php, .jsp, .asp, .cgi) rather than alerting on all file writes, since legitimate deployments regularly write static assets.
- Exclude the application's own deployment or CI/CD process by process ancestry (e.g. writes originating from a known deployment pipeline user or service account) so that routine releases are not flagged.
- Where feasible, correlate with web server access logs for a subsequent HTTP request to the newly created file, which substantially increases confidence that the file is being actively used as a web shell rather than sitting unused.

5. Validation Results

The table below summarizes the observed outcome of each attack simulation. “Observed Evidence” refers strictly to the source telemetry captured from Suricata or auditd; “Wazuh Alert” refers strictly to the alert generated by the Wazuh rule engine after decoding and rule evaluation. Results reflect what was directly observed in this lab environment and are not extrapolated beyond the documented test conditions.

Detection Name	Attack Simulation	Expected Behavior	Observed Evidence	Wazuh Alert	Severity	MITRE Technique
Nmap SYN Scan Detection	nmap -sS against target host	Suricata signature matches repeated SYN packets from one source; Wazuh escalates on frequency threshold	Suricata eve.json alert, signature_id 9000001, src_ip 192.168.0.122	Rule 100011 fired after frequency condition met (5/120s)	Level 10	T1046
Base64 Encoded Command Execution	echo <b64> base64 -d bash	auditd captures execve of base64 with monitored key; Wazuh alerts on single match	auditd SYSCALL/EXECVE, key=base64_exec, exe=/usr/bin/base64	Rule 100020 fired on audit.key match	Level 7	T1027
Suspicious User Creation	useradd -m backupsvc	auditd captures execve of useradd with monitored key; Wazuh alerts on single match	auditd SYSCALL/EXECVE, key=user_creation, exe=/usr/sbin/useradd, a0=useradd	Rule 100030 fired on audit.key + audit.exe match	Level 8	T1136
Web Shell Upload Detection	tee writes shell_test.php into /var/www/html	auditd captures PATH CREATE event under web root with monitored key; Wazuh escalates on .php extension match	auditd PATH/SYSCALL, key=webroot_changes, name=/var/www/html/shell_test.php, nametype=CREATE, exe=/usr/bin/tee	Rule 100041 fired on nametype=CREATE + .php regex match	Level 12	T1505.003

All four rules produced an alert consistent with their documented matching logic during testing. No unexpected alert suppression or duplicate firing was observed for the specific test cases above; broader noise characteristics under sustained normal usage were not measured in this lab and are addressed as an open tuning consideration in Section 6 rather than a validated result.

6. Detection Tuning

6.2 Parent-Child Process Relationships

Where auditd data includes process ancestry, tuning can be extended by requiring that the monitored action originate from an unexpected parent process. For example, a base64-decode-into-shell pattern (Detection 2) originating from a user's interactive shell is a different risk profile than the same pattern originating from a known configuration-management agent; the latter can be deprioritized or excluded by matching on the parent process field rather than the executed command alone. This project's rules are written to be extended with such a condition but do not currently enforce it, since the lab's audit configuration was scoped to syscall and key matching rather than full process-tree capture.

6.3 Monitoring Sensitive Directories Only

Detection 4 deliberately scopes file monitoring to the web document root rather than the filesystem broadly. This is a direct application of the principle that behavioral monitoring should be concentrated on directories with a narrow, well-understood legitimate write pattern (deployment processes, specific service accounts) so that any write outside that pattern is inherently higher-signal. The same principle should be applied to any future detections targeting other sensitive paths (e.g. cron directories, SSH authorized_keys, systemd unit directories).

6.4 Whitelisting Trusted Administrators

Several detections in this report (base64 execution, user creation) have legitimate administrative use cases. The recommended tuning approach is not to lower rule severity across the board, but to maintain an explicit allow-list of accounts or service identities with a documented, recurring operational need to perform the monitored action, and to suppress or downgrade only alerts tied to that allow-list – preserving full-severity alerting for any other account performing the same action.

6.5 Frequency and Timeframe Correlation

Frequency/timeframe correlation was used specifically for the Nmap SYN scan detection, where a single match is common and only a sustained pattern is meaningful. It was deliberately not used for base64 execution, user creation, or web shell upload, where a single occurrence already carries sufficient signal and waiting for repetition would delay detection of a fast, single-action attack. This distinction – correlating only where the underlying behavior is naturally noisy in isolation – was a core design decision across the project rather than a default applied uniformly.

6.6 Avoiding Broad Signatures

Each custom rule in this project matches on a specific, purpose-built audit key or Suricata signature_id rather than a generic string pattern (e.g. matching literal command text, or matching on any Suricata alert regardless of signature). This keeps each rule's false-positive surface tied directly to the audit or IDS configuration that produces the underlying telemetry, and makes the rule's intent traceable back to a single, documented source event rather than an opaque regular expression.

7. Conclusion

This project implemented four custom Wazuh detection rules addressing gaps in default coverage across reconnaissance, defense evasion, persistence, and initial-access behaviors. Each rule was built on a documented data source (Suricata eve.json or Linux auditd), validated against a controlled attack simulation, and mapped to a specific MITRE ATT&CK technique to support downstream triage and reporting.

The detection engineering approach applied throughout – a low-level base rule confirming decoder output, followed by a more specific child rule carrying the operational alert – proved effective for keeping individual rules simple while still supporting escalation logic where needed (Detections 1 and 4). Where a single event was already high-signal (Detections 2 and 3), a single-tier rule was used instead, avoiding unnecessary complexity.

Validation confirmed that all four rules produced Wazuh alerts consistent with their intended matching logic under the tested conditions. This report does not claim these detections are complete, exhaustive, or free of false positives in a production environment; Section 6 documents the specific tuning steps – field-combination scoping, process-ancestry awareness, directory scoping, administrator allow-listing, and selective frequency correlation – that would be required before deployment outside a controlled lab. Future work could extend this project by incorporating process-tree context into the auditd rules, expanding web-directory monitoring to additional script extensions and frameworks, and building corresponding active-response actions in Wazuh for the highest-severity alerts (Detection 4 in particular).